

# GPUMODE Lecture Study Notes: Lecture 63

## Search Based Deep Learning Compilers

### Core Problem the Lecture Addresses

This lecture explains a search-first approach to deep learning compilation, using Luminal as the central example. The key idea is that modern machine learning systems are enormously complicated because they combine many operations, many data types, many hardware backends, many handcrafted kernels, and many runtime execution policies. The lecture argues that this complexity can be reduced by representing neural networks with a very small set of primitive operations and then using compiler search to recover high performance.

The main technical problem is:

**Can we replace a huge library of handcrafted kernels with a small primitive IR plus search?**

In conventional systems, a framework such as PyTorch exposes a large operation set. If there are  $O$  operations,  $D$  data types, and  $H$  hardware targets, the implementation burden behaves roughly like

$$\text{Implementation complexity} \approx O \cdot D \cdot H.$$

This multiplicative scaling creates an explosion in engineering work. Adding one new device means supporting many operations and many data types. Adding one new operation means supporting many data types and many devices.

Luminal instead takes the opposite direction:

Large API surface  $\longrightarrow$  small primitive graph  $\longrightarrow$  search over equivalent low-level programs.

The lecture's thesis is that a deep learning compiler can be radically simplified if the input language is restricted enough. Neural networks are mostly tensor algebra, and many high-level operations can be decomposed into a small number

of primitives. The cost is that the naive primitive graph is slow. The compiler’s job is to transform that slow graph into fast kernels.

The mental shift is: do not handwrite every kernel. Write a small set of primitives, describe legal rewrites, generate many equivalent kernels, profile them, and keep the best one.

## Required Background

### Deep Learning as Tensor Algebra

Most modern neural networks can be expressed as computations over scalars, vectors, matrices, and higher-dimensional tensors. The fundamental objects are tensors:

$$X \in \mathbb{R}^{d_0 \times d_1 \times \cdots \times d_{n-1}}.$$

A tensor operation maps one or more input tensors to one or more output tensors. For example, an elementwise unary operation has the form

$$Y_i = f(X_i),$$

where  $i$  indexes tensor elements. An elementwise binary operation has the form

$$Z_i = f(X_i, Y_i).$$

A reduction collapses one or more dimensions:

$$Y_{i,j} = \sum_k X_{i,j,k}.$$

Matrix multiplication can be expressed as a broadcasted multiply followed by a reduction:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

This is important because a compiler does not necessarily need a primitive matrix multiplication operation. It can build matrix multiplication from simpler primitives, then later rediscover efficient matrix multiplication implementations through transformations.

## Static Computation Graphs

Luminal assumes that many deployed neural networks are mostly static. The weights and activations change, but the graph structure usually does not. This differs from the original PyTorch design philosophy, where dynamic execution was a major feature.

A static computation graph is a directed acyclic graph:

$$G = (V, E),$$

where each node  $v \in V$  is an operation and each edge  $e \in E$  carries a tensor from one operation to another.

For inference, this assumption is especially natural. A transformer layer, for example, has a fixed sequence of tensor operations. The input tokens, activations, and weights vary, but the graph topology is usually known ahead of time.

## GPU Execution Model

A CUDA program launches kernels over a grid of thread blocks. A simplified hierarchy is:

$$\text{Grid} \rightarrow \text{Thread Blocks} \rightarrow \text{Warps} \rightarrow \text{Threads}.$$

Important GPU concepts include:

- **Streaming multiprocessor:** an SM executes warps and contains registers, shared memory, warp schedulers, execution units, and load-store units.
- **Warp:** a group of usually 32 threads that execute instructions in SIMT fashion.
- **Global memory:** large off-chip memory, high bandwidth but high latency.
- **Shared memory:** on-chip memory local to a thread block, lower latency than global memory.
- **Registers:** per-thread storage, fastest but limited.
- **Occupancy:** how many warps or thread blocks can be resident on an SM.
- **Memory coalescing:** arranging adjacent threads to access adjacent memory locations.
- **Tensor cores:** specialized matrix-multiply units for low precision and mixed precision operations.

The lecture repeatedly returns to a core performance principle:

GPU performance is often limited by data movement, not arithmetic.

## Compilers, IRs, and Rewrites

A compiler transforms a high-level representation into a lower-level representation. In this lecture, the high-level representation is a primitive tensor graph. The lower-level representation is generated CUDA, Metal, CPU code, or another backend.

An intermediate representation, or IR, is the internal program representation used by the compiler. Luminal represents computations using loop nodes, load nodes, operation nodes, and store nodes.

A rewrite rule is a local equivalence rule:

$$\text{pattern}_{\text{before}} \iff \text{pattern}_{\text{after}}.$$

For example, the algebraic identity

$$x \div 1 = x$$

can be used as a rewrite rule that eliminates unnecessary division.

## Main Concepts

### Minimal Primitive Operation Set

Luminal deliberately avoids a large operation set. Instead of exposing hundreds or thousands of operations, it uses a small number of primitive tensor operations.

The lecture mentions primitive categories such as:

- **Unary elementwise operations:** exponential base 2, logarithm base 2, sine, reciprocal, square root.
- **Binary elementwise operations:** add, multiply, modulo, less-than.
- **Reductions:** sum reduce and max reduce.

The key design goal is not mathematical minimality in the theoretical sense. In principle, many computations can be reduced to an extremely tiny basis such as Boolean operations. But that would make graphs enormous and rewrite matching impractical. Luminal chooses primitives that are still broadly useful for neural networks.

The design trade-off is:

Too many primitives  $\Rightarrow$  large framework and backend complexity,

Too few primitives  $\Rightarrow$  huge graphs and expensive search.

Luminal chooses a middle point:

small enough to simplify the compiler, large enough to keep graphs searchable.

## Derived Operations

Many familiar operations can be expressed using primitives.

### Subtraction

Subtraction can be expressed as addition and multiplication:

$$a - b = a + (-1)b.$$

Thus a separate subtraction primitive is unnecessary.

### Division

Division can be expressed using reciprocal and multiplication:

$$\frac{a}{b} = a \cdot \frac{1}{b}.$$

If reciprocal is a primitive unary operation, division becomes a derived binary expression.

### Matrix Multiplication

Matrix multiplication is a broadcasted multiply followed by a sum reduction:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

The primitive graph form is:

$$A_{m,k}, B_{k,n} \longrightarrow A_{m,k} B_{k,n} \longrightarrow \sum_k A_{m,k} B_{k,n}.$$

This is not initially fast, but it is semantically simple. Later compiler rewrites can fuse loops, tile loops, and eventually map suitable patterns onto tensor core instructions.

## Convolution

A convolution can be expressed through indexing, local window extraction, multiplication, and reduction. A simplified two-dimensional convolution is

$$Y_{n,c_{\text{out}},h,w} = \sum_{c_{\text{in}}} \sum_r \sum_s X_{n,c_{\text{in}},h+r,w+s} W_{c_{\text{out}},c_{\text{in}},r,s}.$$

Conceptually, convolution is a structured gather plus multiply-reduce. If the compiler can represent the indexing and reduction structure, convolution does not require a special primitive.

## Static Graphs Instead of Dynamic Execution

The lecture emphasizes that PyTorch’s dynamism is excellent for development but costly for compilation and deployment. Dynamic execution means the runtime must remain flexible. Static graphs allow more aggressive ahead-of-time optimization.

For a static graph, the compiler can decide ahead of time:

- which operations can be fused,
- which loops can be tiled,
- which buffers can be reused,
- where memory transfers occur,
- which shapes should be specialized,
- which symbolic dimensions should remain dynamic,
- which hardware intrinsics should be used.

A model graph may be static even when its data is dynamic. Let  $\theta$  denote model weights and  $x$  denote activations. The graph computes

$$y = f_G(x; \theta),$$

where  $G$  is fixed but  $x$  and  $\theta$  may vary.

## Symbolic Dimensions

A major practical issue in compilation is shape specialization. If every dimension is compiled as a fixed integer, then changing batch size or sequence length may require recompilation. Luminal supports symbolic dimensions.

For example, instead of compiling for a fixed batch size  $B = 10$ , the graph can contain a symbolic batch dimension:

$$B \in \mathbb{N}.$$

A kernel can then accept  $B$  as a runtime value while still being compiled from the same graph. This is useful for dynamic batch sizes, variable sequence lengths, and serving workloads with changing request patterns.

The trade-off is:

fully static shapes  $\Rightarrow$  more specialization and potentially faster kernels,

symbolic shapes  $\Rightarrow$  more flexibility and fewer recompilations.

A practical system can profile representative concrete values while still producing a symbolically valid kernel. For example, it can search using  $B = 1$  and  $B = 64$  as representative cases while preserving a symbolic batch dimension in the generated code.

## Search as Autotuning on Many Axes

Traditional autotuning often searches over a fixed set of parameters:

tile size, num warps, block size, unroll factor.

Luminal generalizes this idea. Search is not only over numeric tuning knobs but also over program structure:

Should loops be fused?

Should loops be split?

Should a loop be tiled?

Should an intermediate buffer be materialized?

Should an algebraic rewrite be applied?

Should a tensor core intrinsic replace a multiply-reduce pattern?

Thus, the lecture describes Luminal as an autotuner over a much richer space of equivalent programs.

## The Search Space of Equivalent Kernels

Given a primitive computation graph  $G_0$ , rewrite rules generate a set of equivalent graphs:

$$\mathcal{S}(G_0) = \{G_1, G_2, \dots, G_N\}.$$

Each graph should compute the same mathematical result, up to allowed floating-point tolerance. Each graph can be lowered to one or more concrete kernels.

A profiler measures runtime:

$$T(G_i) = \text{measured runtime of generated kernel for } G_i.$$

The compiler chooses

$$G^\star = \arg \min_{G_i \in \mathcal{S}(G_0)} T(G_i).$$

The simplest search strategy is brute force: generate many candidate kernels, compile them, run them, validate outputs, and keep the fastest. More advanced methods include beam search, Monte Carlo tree search, learned-guided search, and pruning.

## Numerical Correctness and Floating-Point Tolerance

Mathematical equivalence is not always the same as floating-point equivalence. For example, associativity holds over real numbers:

$$(a + b) + c = a + (b + c),$$

but floating-point addition is not exactly associative:

$$\text{fl}(\text{fl}(a + b) + c) \neq \text{fl}(a + \text{fl}(b + c)).$$

Luminal currently handles this by validating candidate kernels against reference outputs within a tolerance. A typical validation condition is

$$\|Y_{\text{candidate}} - Y_{\text{reference}}\|_\infty \leq \epsilon,$$

or a relative tolerance condition such as

$$\frac{\|Y_{\text{candidate}} - Y_{\text{reference}}\|_\infty}{\max(\|Y_{\text{reference}}\|_\infty, \delta)} \leq \epsilon.$$



The lecture notes that a future improvement would be symbolic or static reasoning about numerical error, so the compiler could reject unstable rewrites before profiling.

## Hardware-Level Explanation

### Why Primitive Graphs Are Slow by Default

A primitive graph may represent a high-level operation using many tiny operations. If executed naively, each primitive may become a separate kernel. This causes severe overhead:

$$\text{Total time} = \sum_i (T_{\text{launch},i} + T_{\text{memory},i} + T_{\text{compute},i}).$$

For simple elementwise operations,  $T_{\text{compute}}$  is often tiny compared with memory movement and launch overhead. If every primitive writes its output to global memory and the next primitive reads it back, the program becomes bandwidth-bound and launch-bound.

For example, consider:

$$Y = \exp_2(\sin(X)).$$

An unfused execution might perform:

$$X \rightarrow \sin(X) \rightarrow \text{global memory} \rightarrow \exp_2(\sin(X)) \rightarrow \text{global memory}.$$

This causes an unnecessary round trip through global memory. A fused kernel does:

$$X \rightarrow \sin(X) \rightarrow \exp_2(\sin(X)) \rightarrow Y$$

inside one kernel, keeping the intermediate value in registers.

### Kernel Fusion

Kernel fusion reduces memory movement by combining operations into one kernel.

Suppose  $X$  has  $N$  elements and each element is a 4-byte floating-point value. Two separate elementwise kernels may read and write:

$$\text{Bytes}_{\text{unfused}} = \underbrace{4N}_{\text{read } X} + \underbrace{4N}_{\text{write temp}} + \underbrace{4N}_{\text{read temp}} + \underbrace{4N}_{\text{write } Y} = 16N.$$

A fused kernel reads  $X$  once and writes  $Y$  once:

$$\text{Bytes}_{\text{fused}} = \underbrace{4N}_{\text{read } X} + \underbrace{4N}_{\text{write } Y} = 8N.$$

Thus, ideal fusion cuts global memory traffic by approximately a factor of 2:

$$\frac{\text{Bytes}_{\text{unfused}}}{\text{Bytes}_{\text{fused}}} = \frac{16N}{8N} = 2.$$

This is why elementwise fusion often produces large speedups on GPUs.

### Visual Concept

Draw two diagrams side by side. In the unfused version, show high-bandwidth memory sending data to an SM, the SM writing an intermediate back to memory, then reading it again. In the fused version, show one read from memory, multiple operations inside the SM register file, and one final write.

## Memory Hierarchy and Data Movement

A simplified GPU memory hierarchy is:

Registers  $\rightarrow$  Shared Memory  $\rightarrow$  L1 Cache  $\rightarrow$  L2 Cache  $\rightarrow$  Global Memory.

Performance improves when data moves upward in this hierarchy and is reused before being evicted. Search-based compilation can discover transformations that increase reuse:

- fuse kernels to keep intermediates in registers,
- tile loops to reuse data in shared memory or registers,
- reorder loops to improve memory coalescing,
- avoid materializing temporary tensors,
- use tensor cores when the shape and data type match hardware requirements.

## Streaming Multiprocessors and Fused Elementwise Kernels

In a fused elementwise kernel, each thread commonly handles one or more elements:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

The thread reads  $X_i$ , computes a chain of scalar operations in registers, and writes  $Y_i$ . If adjacent threads access adjacent addresses, the memory accesses are coalesced.

The hardware behavior is:

- global load brings  $X_i$  into registers,
- arithmetic instructions execute on SM scalar or special-function units,
- intermediate values stay in registers,
- global store writes  $Y_i$ ,
- no shared memory is needed for purely elementwise work.

## Matrix Multiplication and Accumulation

For matrix multiplication,

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n},$$

a naive kernel may assign one output element  $C_{m,n}$  to one thread. That thread loops over  $k$  sequentially and accumulates into a register:

$$\text{acc}_{m,n}^{(0)} = 0,$$

$$\text{acc}_{m,n}^{(k+1)} = \text{acc}_{m,n}^{(k)} + A_{m,k} B_{k,n}.$$

After  $K$  iterations,

$$C_{m,n} = \text{acc}_{m,n}^{(K)}.$$

This naive approach is simple but inefficient because it does not reuse loaded  $A$  and  $B$  values across multiple output elements. A tiled matrix multiplication improves reuse by loading blocks of  $A$  and  $B$  into shared memory and computing a tile of  $C$ .

The search-based view is that both naive and tiled versions are points in the same equivalence space. Loop tiling, loop splitting, loop fusion, and tensor core insertion can move from one point to another.

## Tensor Core Intrinsic as Rewrite Targets

Hardware intrinsics can also appear as rewrite targets. For example, if the graph contains a pattern equivalent to a small matrix multiply whose dimensions match a tensor core instruction, the compiler can rewrite that pattern to a tensor core operation.

An abstract rule is:

multiply-reduce tile of shape  $M \times N \times K \longrightarrow$  tensor core MMA intrinsic.

This rewrite is device-specific. A rule suitable for one NVIDIA generation may not map directly to another architecture. Hopper and Blackwell introduce increasingly complex hardware mechanisms, such as asynchronous memory movement and more explicit producer-consumer patterns. The lecture argues that search becomes more valuable as hardware complexity increases.

## Static Scheduling and the VLIW Analogy

The lecture connects ML compilation to the idea of very long instruction word architectures. VLIW tries to move scheduling complexity into the compiler. The hardware becomes simpler because the compiler decides what can run in parallel.

The historical problem is the VLIW paradox:

general-purpose program complexity  $\Rightarrow$  extremely complex compiler.

For arbitrary programs, the compiler must handle almost anything. For neural networks, the input domain is narrower. This makes search-based static scheduling more plausible.

The core argument is:

restricted ML graphs+search  $\Rightarrow$  compiler can handle complexity that would be hard for a general-purpose compiler.

## Mathematical Reasoning

### Complexity Scaling of Traditional Frameworks

Let

$O$  = number of operations,

$D$  = number of data types,

$H$  = number of hardware targets.

A rough implementation count is

$$I_{\text{traditional}} = ODH.$$

If  $O = 1200$ ,  $D = 15$ , and  $H = 5$ , then

$$I_{\text{traditional}} = 1200 \cdot 15 \cdot 5 = 90000.$$

This is not a literal count of source files, but it captures the combinatorial pressure. Every axis multiplies the maintenance burden.

Luminal instead tries to reduce  $O$  dramatically:

$$I_{\text{primitive}} = PDH,$$

where  $P$  is the primitive operation count and  $P \ll O$ . The missing complexity is shifted into compiler search and rewrite rules.

## Arithmetic Intensity

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

A kernel is likely memory-bound if

$$\frac{\text{FLOPs}}{\text{Bytes moved}} < \frac{\text{Peak FLOPs/s}}{\text{Peak Bytes/s}}.$$

It is likely compute-bound if

$$\frac{\text{FLOPs}}{\text{Bytes moved}} > \frac{\text{Peak FLOPs/s}}{\text{Peak Bytes/s}}.$$

Elementwise kernels have low arithmetic intensity. For one unary operation on FP32 data, a simple model is:

$$\text{FLOPs} \approx N,$$

$$\text{Bytes moved} \approx 8N,$$

because each element is read once and written once. Thus,

$$\text{Arithmetic Intensity} \approx \frac{N}{8N} = \frac{1}{8} \text{ FLOPs/byte.}$$

This is usually memory-bound.

For matrix multiplication  $C = AB$  with square matrices of size  $N \times N$ , the FLOP count is approximately

$$\text{FLOPs} \approx 2N^3.$$

If each matrix is read or written once, the minimal memory traffic is roughly

$$\text{Bytes} \approx 3N^2s,$$

where  $s$  is bytes per element. The ideal arithmetic intensity is

$$\text{AI}_{\text{matmul}} \approx \frac{2N^3}{3N^2s} = \frac{2N}{3s}.$$

For FP16,  $s = 2$ , so

$$\text{AI}_{\text{matmul}} \approx \frac{N}{3}.$$

This grows with  $N$ , which is why large matrix multiplication can become compute-bound if implemented with sufficient data reuse.

## Roofline Model

The roofline performance model says

$$P_{\text{achieved}} \leq \min(P_{\text{peak}}, B_{\text{mem}} \cdot \text{AI}),$$

where:

- $P_{\text{achieved}}$  is achieved throughput,
- $P_{\text{peak}}$  is peak compute throughput,

- $B_{\text{mem}}$  is memory bandwidth,
- AI is arithmetic intensity.

Compiler rewrites can improve performance by:

decreasing bytes moved  $\Rightarrow$  increasing arithmetic intensity,

or by

using tensor cores  $\Rightarrow$  increasing effective compute throughput.

## Loop Fusion as a Memory Traffic Transformation

Given two elementwise functions,

$$T_i = f(X_i),$$

$$Y_i = g(T_i),$$

the unfused execution materializes  $T$ . The fused execution computes

$$Y_i = g(f(X_i))$$

directly.

The mathematical expression is the same, but the memory behavior changes. Fusion is therefore both an algebraic and a systems optimization:

same function  $\neq$  same memory traffic.

## Loop Splitting and Tiling

A loop

**for**  $i = 0, \dots, N - 1$

can be split into nested loops using tile size  $T$ :

$$i = i_{\text{outer}}T + i_{\text{inner}},$$

where

$$i_{\text{outer}} = 0, \dots, \left\lceil \frac{N}{T} \right\rceil - 1,$$

$$i_{\text{inner}} = 0, \dots, T - 1.$$

This transformation exposes locality. In a GPU kernel,  $i_{\text{outer}}$  may map to thread blocks and  $i_{\text{inner}}$  may map to threads, vector lanes, registers, or shared memory tiles.

For matrix multiplication, tiling decomposes

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}$$

into tile-level accumulation:

$$C_{m,n} = \sum_{t=0}^{\lceil K/T_K \rceil - 1} \sum_{r=0}^{T_K - 1} A_{m, tT_K + r} B_{tT_K + r, n}.$$

This creates reuse opportunities for  $A$  and  $B$  tiles.

## Online Softmax and Flash Attention

Naive attention computes

$$S = QK^{\top},$$

$$P = \text{softmax}(S),$$

$$O = PV.$$

For one query row, the softmax is

$$p_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}.$$

A numerically stable form subtracts the maximum:

$$p_i = \frac{\exp(s_i - m)}{\sum_j \exp(s_j - m)},$$



where

$$m = \max_j s_j.$$

Flash attention avoids materializing the full attention matrix  $S$  and probability matrix  $P$ . It computes attention block by block while maintaining an online maximum and normalization factor.

Suppose after processing some blocks, we maintain:

$$m_{\text{old}} = \max \text{ of seen scores,}$$

$$\ell_{\text{old}} = \sum_{\text{seen } j} \exp(s_j - m_{\text{old}}),$$

$$o_{\text{old}} = \sum_{\text{seen } j} \exp(s_j - m_{\text{old}}) v_j.$$

For a new block, define

$$m_{\text{block}} = \max_{\text{new } j} s_j,$$

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}}).$$

The old normalizer must be rescaled:

$$\ell'_{\text{old}} = \ell_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}).$$

The new block contribution is

$$\ell_{\text{block}} = \sum_{\text{new } j} \exp(s_j - m_{\text{new}}).$$

The updated normalizer is

$$\ell_{\text{new}} = \ell'_{\text{old}} + \ell_{\text{block}}.$$

The output accumulator is similarly updated:

$$o_{\text{new}} = o_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}) + \sum_{\text{new } j} \exp(s_j - m_{\text{new}}) v_j.$$

The final output is

$$O = \frac{O_{\text{final}}}{\ell_{\text{final}}}.$$

The lecture states that Luminal's search was able to discover a flash-attention-like implementation from a naive attention graph using a sequence of generic rewrites plus an online softmax or online max rule.

## Code Walkthrough

### Representative Fused Elementwise CUDA Kernel

The lecture shows a simple IR where a vector is loaded, transformed by operations such as exponential and sine, then stored. A representative fused CUDA kernel is:

```
extern "C" __global__
void fused_exp2_sin_kernel(const float* A, float* B, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        float x = A[idx];
        float y = exp2f(x);
        float z = sinf(y);
        B[idx] = z;
    }
}
```

This kernel maps one thread to one tensor element. The index calculation

```
int idx = blockIdx.x * blockDim.x + threadIdx.x;
```

maps the CUDA grid to a linear element index. If adjacent threads have adjacent `idx` values, the load

```
float x = A[idx];
```

is coalesced. The intermediate values `x`, `y`, and `z` live in registers. The kernel writes only the final result:

```
B[idx] = z;
```

No global memory temporary is created.

### Representative Unfused Version

An unfused version would launch two kernels and materialize an intermediate buffer.

```

extern "C" __global__
void exp2_kernel(const float* A, float* Temp, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        Temp[idx] = exp2f(A[idx]);
    }
}

extern "C" __global__
void sin_kernel(const float* Temp, float* B, int N) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (idx < N) {
        B[idx] = sinf(Temp[idx]);
    }
}

```

This version performs an extra global memory write and read. It also pays two kernel launches instead of one. For low arithmetic intensity operations, this is usually much slower.

## Representative Naive Matrix Multiplication Kernel

The lecture explains a naive matrix multiplication IR with loop-ins over  $M$ ,  $N$ , and  $K$ , where the  $K$  loop is sequential accumulation. A representative CUDA kernel is:

```

extern "C" __global__
void naive_matmul_kernel(
    const float* A,
    const float* B,
    float* C,
    int M,
    int N,
    int K
) {
    int n = blockIdx.x * blockDim.x + threadIdx.x;
    int m = blockIdx.y * blockDim.y + threadIdx.y;

    if (m < M && n < N) {
        float acc = 0.0f;

        for (int k = 0; k < K; ++k) {
            float a = A[m * K + k];
            float b = B[k * N + n];
            acc += a * b;
        }
    }
}

```

```

    }

    C[m * N + n] = acc;
}
}

```

The indexing assumes row-major layout:

$$A_{m,k} \leftrightarrow A[m * K + k],$$

$$B_{k,n} \leftrightarrow B[k * N + n],$$

$$C_{m,n} \leftrightarrow C[m * N + n].$$

The inner loop

```
for (int k = 0; k < K; ++k)
```

is a sequential accumulation loop. The accumulator `acc` is stored in a register. The code is simple but inefficient because each thread independently loads its own  $A$  and  $B$  values without cooperative reuse.

## Representative Tiled Matrix Multiplication Kernel

A more efficient version uses shared memory tiling:

```

#define TILE 16

extern "C" __global__
void tiled_matmul_kernel(
    const float* A,
    const float* B,
    float* C,
    int M,
    int N,
    int K
) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int tx = threadIdx.x;
    int ty = threadIdx.y;

    int n = blockIdx.x * TILE + tx;
    int m = blockIdx.y * TILE + ty;

```

```

float acc = 0.0f;

for (int tile = 0; tile < (K + TILE - 1) / TILE; ++tile) {
    int a_col = tile * TILE + tx;
    int b_row = tile * TILE + ty;

    if (m < M && a_col < K) {
        As[ty][tx] = A[m * K + a_col];
    } else {
        As[ty][tx] = 0.0f;
    }

    if (b_row < K && n < N) {
        Bs[ty][tx] = B[b_row * N + n];
    } else {
        Bs[ty][tx] = 0.0f;
    }

    __syncthreads();

    for (int k_inner = 0; k_inner < TILE; ++k_inner) {
        acc += As[ty][k_inner] * Bs[k_inner][tx];
    }

    __syncthreads();
}

if (m < M && n < N) {
    C[m * N + n] = acc;
}
}

```

This kernel demonstrates the kind of transformation a compiler may search for:

- split the  $K$  loop into outer and inner tile loops,
- cooperatively load tiles into shared memory,
- synchronize threads after loading,
- reuse each tile for multiple multiply-add operations,
- write the final result once.

The shared memory arrays

```

__shared__ float As[TILE][TILE];
__shared__ float Bs[TILE][TILE];

```

store reusable tiles. The synchronization

```
__syncthreads();
```

is necessary because all threads in the block must finish loading before any thread consumes the shared memory tile.

## Representative Symbolic Shape Kernel

Symbolic dimensions become runtime parameters:

```
extern "C" __global__
void symbolic_elementwise_kernel(
    const float* A,
    float* B,
    int batch,
    int features
) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int total = batch * features;

    if (idx < total) {
        float x = A[idx];
        B[idx] = x * x + 1.0f;
    }
}
```

Here, `batch` and `features` are not fixed at compile time. This allows the same kernel to handle multiple input sizes. A more specialized kernel might hardcode one or both dimensions, enabling more aggressive optimization but requiring recompilation or dispatch among specializations.

## Pseudo-Code for Search-Based Compilation

The lecture describes a pipeline that can be written conceptually as:

```
def compile_with_search(model_graph, rewrite_rules, example_shapes):
    primitive_graph = lower_to_primitives(model_graph)

    egglog_program = convert_to_egglog(primitive_graph)

    search_space = equality_saturate(
        program=egglog_program,
        rules=rewrite_rules,
        iterations=MAX_ITERATIONS
    )

    best_kernel = None
```

```

best_time = float("inf")

for candidate_graph in extract_candidates(search_space):
    kernel_source = codegen(candidate_graph)
    compiled_kernel = compile_kernel(kernel_source)

    for shape in example_shapes:
        outputs = run_kernel(compiled_kernel, shape)
        reference = run_reference(primitive_graph, shape)

        if not within_tolerance(outputs, reference):
            continue

    runtime = profile(compiled_kernel, shape)

    if runtime < best_time:
        best_time = runtime
        best_kernel = compiled_kernel

return best_kernel

```

The important parts are:

- **lower\_to\_primitives**: expresses the model using the small primitive operation set.
- **convert\_to\_egglog**: translates the graph into the equality saturation engine.
- **equality\_saturate**: applies rewrite rules in many orders to create a large equivalence space.
- **extract\_candidates**: selects concrete candidate graphs from the e-graph.
- **codegen**: lowers a candidate graph into CUDA, Metal, CPU code, or another backend.
- **within\_tolerance**: rejects numerically invalid candidates.
- **profile**: measures real hardware runtime.

## E-Graphs, Egglog, and Equality Saturation

### The Ordering Problem

In a traditional compiler, optimization passes are ordered manually:

$$P_0 \rightarrow P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_n.$$

The ordering problem is that the best order may depend on the program. Sometimes applying optimization  $A$  before  $B$  is best. Sometimes  $B$  before  $A$  is best. Sometimes  $A$  must be applied multiple times before  $B$  becomes useful.

Equality saturation avoids committing to one order. It applies many rewrite rules and stores many equivalent expressions compactly.

## E-Graphs

An e-graph is a data structure that compactly stores many equivalent expressions. It consists of e-classes and e-nodes.

An e-class represents a set of equivalent expressions:

$$[e] = \{e_1, e_2, \dots, e_k\}.$$

An e-node represents one operator applied to child e-classes. For example,

$$(a + b) + c$$

and

$$a + (b + c)$$

can belong to the same e-class after applying associativity.

The e-graph can represent exponentially many expressions without explicitly storing each one separately.

## Equality Saturation

Equality saturation repeatedly applies rewrite rules until a limit is reached. The process is:

$$G_0 \rightarrow G_1 \rightarrow G_2 \rightarrow \dots \rightarrow G_T,$$

where each  $G_t$  is an expanded e-graph containing more equivalent expressions than before.

A rewrite rule has the form

$$\text{left pattern} \longrightarrow \text{right pattern}.$$

For associativity:



$$(a + b) + c \longrightarrow a + (b + c).$$

For division by one:

$$x \div 1 \longrightarrow x.$$

For loop fusion, an abstract rule is:

$$\begin{aligned} & \text{loop } i \text{ computing } T_i = f(X_i) \quad + \quad \text{loop } i \text{ computing } Y_i = g(T_i) \\ & \longrightarrow \text{loop } i \text{ computing } Y_i = g(f(X_i)). \end{aligned}$$

## Why Simple Rules Can Produce Complex Optimizations

The lecture emphasizes that Luminal does not need a massive number of rewrite rules. Complex optimizations emerge from sequences of simple rewrites.

Let  $R$  be a set of rewrite rules:

$$R = \{r_1, r_2, \dots, r_m\}.$$

A complex transformation may be a composition:

$$G_{\text{optimized}} = r_{i_k} \circ r_{i_{k-1}} \circ \dots \circ r_{i_1}(G_{\text{naive}}).$$

The programmer does not need to hand-code the whole transformation if the search process can discover the right sequence.

## Flash Attention as an Emergent Search Result

A striking claim in the lecture is that naive attention was converted into a flash-attention-like implementation through search. The compiler started with a naive attention graph and applied a sequence of rewrites such as:

- loop fusion,
- online max or online softmax transformation,
- additional loop fusion,
- loop swapping,
- tiling,
- further loop reordering and simplification.

This is different from conventional pattern matching. A pattern matcher would detect attention and replace it with a handwritten flash attention kernel. Luminal instead expands the search space until a flash-like algorithm appears as one valid point.

The distinction is:

pattern matching : recognize attention and call handcrafted kernel,

search : derive an equivalent algorithm through rewrite composition.

## Intermediate Representation

### Loop-In and Loop-Out Nodes

The lecture describes an IR where loops are explicit. A loop-in represents entering a loop and stepping through input memory. A loop-out represents storing results and exiting the loop.

A loop has an iterator. For example:

$$i = 0, 1, \dots, N - 1.$$

A stride controls how memory is traversed:

$$\text{address}(i) = \text{base} + i \cdot s.$$

If  $s = 1$ , the loop visits contiguous elements. If  $s = 2$ , it visits every second element. If the stride expression changes, the same logical operation can represent transposed, strided, or broadcasted memory access.

### Strides as a Core Abstraction

For a row-major matrix  $X \in \mathbb{R}^{M \times N}$ , the address of element  $X_{m,n}$  is:

$$\text{addr}(X_{m,n}) = \text{base}(X) + mN + n.$$

The strides are:

$$s_m = N, \quad s_n = 1.$$

A transposed view changes the logical indexing without moving data:

$$X_{n,m}^\top = X_{m,n}.$$

Its logical strides become:

$$s_n = 1, \quad s_m = N.$$

This is why stride manipulation is powerful. It allows loop swapping and layout transformations without necessarily copying data.

### Accumulation Stride

The matrix multiplication IR includes a special accumulation stride. In a normal loop, each iteration writes to a different output location. In an accumulation loop, each iteration updates the same accumulator.

For matrix multiplication:

$$\text{acc} \leftarrow 0,$$

$$\text{acc} \leftarrow \text{acc} + A_{m,k} B_{k,n} \quad \text{for each } k.$$

The accumulator is not stepped like an output pointer. It is repeatedly updated. This lets the IR represent sum reduction, max reduction, and more general reductions.

A generalized reduction can be written as:

$$a_0 = \text{identity},$$

$$a_{k+1} = \phi(a_k, x_k),$$

$$y = a_K.$$

For sum reduction:

$$\phi(a, x) = a + x.$$

For max reduction:

$$\phi(a, x) = \max(a, x).$$

For a more complicated custom reduction,  $\phi$  can be a more complex expression.

## Performance Intuition

### Why Search Helps

Modern GPU optimization has many interacting decisions:

- Which loops should be fused?
- Which loops should be split?
- What tile sizes should be used?
- Should intermediate values be stored in registers, shared memory, or global memory?
- Should the code use scalar CUDA cores or tensor cores?
- Should a shape be specialized or symbolic?
- Should a loop be unrolled?
- Should a kernel be split to reduce register pressure?
- Should a rewrite be skipped for numerical stability?

Human experts can reason through these decisions, but the space grows quickly. Search is useful because it can test many variants empirically.

The optimization objective is not always obvious from static heuristics. A transformation that reduces memory traffic may increase register pressure. A tile size that improves reuse may reduce occupancy. A tensor core rewrite may require data layout conversions that offset its benefit. Search allows the compiler to rely on measured runtime.

### The 90-10 Compile-Time Trade-Off

The lecture notes that a large fraction of speedup often appears early in the search. A rough model is:

first 10% of search time  $\Rightarrow$  about 90% of performance gain.

The remaining gain may require much more search. This is typical of optimization spaces: many simple transformations produce large improvements, while the final few percent require extensive tuning.

This matters because production inference can justify expensive compilation:

$$\text{Total cost} = T_{\text{compile}} + N_{\text{runs}} T_{\text{run}}.$$

A longer compilation is worthwhile if it reduces  $T_{\text{run}}$  enough for large  $N_{\text{runs}}$ :

$$T_{\text{compile,new}} + N_{\text{runs}} T_{\text{run,new}} < T_{\text{compile,old}} + N_{\text{runs}} T_{\text{run,old}}.$$

For high-volume serving, even small runtime improvements can justify minutes or hours of compilation.

## Chunking Large Models

Jointly searching an entire large model may be too expensive. For a transformer with many repeated layers, the compiler can chunk the graph:

$$G = G_{\text{embedding}} \circ G_{\text{layer}}^{(1)} \circ G_{\text{layer}}^{(2)} \circ \dots \circ G_{\text{layer}}^{(L)} \circ G_{\text{head}}.$$

If each layer has the same structure, the compiler can search one representative layer and reuse the result. This reduces compile time dramatically.

The trade-off is that chunk boundaries are hints. They may prevent cross-layer optimizations. Future search methods such as Monte Carlo tree search may reduce the need for such boundaries.

## Monte Carlo Tree Search and Pruning

Brute force search may be expensive. Monte Carlo tree search can explore promising parts of the rewrite space. Instead of enumerating all candidates, it samples rewrite sequences and uses runtime feedback to guide exploration.

A generic search objective is:

$$G^* = \arg \min_G T(G).$$

MCTS approximates this by balancing exploration and exploitation. A common selection score has the form

$$\text{score}(a) = Q(a) + c \sqrt{\frac{\log N}{N(a)}},$$

where:

- $Q(a)$  is the estimated quality of action  $a$ ,
- $N$  is the number of visits to the parent state,
- $N(a)$  is the number of times action  $a$  has been tried,
- $c$  controls exploration.

In compiler search, an action might be applying a rewrite, choosing a tile size, or selecting a hardware intrinsic.

## Deterministic Post-Search Optimizations

Not everything belongs in the search space. Some transformations are almost always beneficial and can be applied deterministically after search.

The lecture mentions:

- **buffer reuse**: if a buffer is no longer needed, its memory can be reused for another buffer,
- **kernel queuing**: reduce CPU dispatch overhead by batching or queuing launches,
- **runtime scheduling improvements**: analogous to CUDA graphs in spirit.

Buffer reuse can be modeled by liveness intervals. If buffer  $A$  is live over interval  $[s_A, e_A]$  and buffer  $B$  is live over  $[s_B, e_B]$ , then they can share storage if

$$e_A < s_B \quad \text{or} \quad e_B < s_A.$$

This is similar to register allocation in traditional compilers.

## Hardware Intrinsic Should Be Searchable

The lecture distinguishes between required system operations and optional hardware intrinsics.

Required operations include moving data to the device or from the device. These must occur somewhere. Optional operations include tensor core intrinsics or architecture-specific instructions. These should be included in the search space because they may or may not improve performance depending on shape, layout, dtype, and surrounding code.

For example:

use tensor core    versus    use scalar CUDA cores

is a searchable choice. The compiler can generate both variants and profile them.

## Training as Compiled Forward and Backward Graphs

### Training as Extended Inference

Although Luminal was originally designed for inference, the lecture explains that training can be viewed as a more complicated graph transformation. Given a forward graph

$$y = f(x; \theta),$$

training requires gradients:

$$\frac{\partial L}{\partial \theta}, \quad \frac{\partial L}{\partial x}.$$

A small autograd engine can derive a backward graph from the primitive forward graph. Because the primitive operation set is small, each primitive needs a derivative rule.

For example, for addition:

$$z = x + y,$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z}, \quad \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z}.$$

For multiplication:

$$z = xy,$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} y, \quad \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} x.$$

For square root:

$$z = \sqrt{x},$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{1}{2\sqrt{x}}.$$

## Forward-Backward Fusion

After deriving the backward graph, the compiler can fuse the forward and backward graphs before running the rest of the optimization pipeline. This means the compiler does not need a completely separate training compiler.

The pipeline is:

$$G_{\text{forward}} \rightarrow G_{\text{backward}} \rightarrow G_{\text{forward-backward}} \rightarrow \text{search and codegen}.$$

This is powerful because training support can be added as an external extension rather than as a large core feature.

## Distributed and Multi-Device Future Direction

### Logical Graphs Across Devices

The lecture discusses future support for distributed execution. The idea is to keep the computation graph logical and let the compiler decide where device boundaries should go.

For two devices, a simple partitioning problem is:

$$G = G_A \cup G_B,$$

where  $G_A$  runs on device  $A$  and  $G_B$  runs on device  $B$ . Edges crossing the partition require communication.

The total runtime model includes compute and communication:

$$T_{\text{total}} = T_{\text{compute},A} + T_{\text{compute},B} + T_{\text{communication}} + T_{\text{synchronization}}.$$

The compiler can search over partition choices:

$$P^* = \arg \min_P T_{\text{total}}(P).$$

### Tensor Parallelism, Pipeline Parallelism, and Data Parallelism

For large models, common distributed strategies include:

- **data parallelism**: replicate the model and split batches,
- **tensor parallelism**: split individual tensor operations across devices,
- **pipeline parallelism**: split layers across devices,
- **expert parallelism**: route mixture-of-experts experts to different devices.

Search could decide which topology fits a particular device cluster, latency model, memory capacity, and workload distribution.

### Collective Operations

Distributed training and inference often require collectives:

all-reduce, all-gather, reduce-scatter, broadcast.

For example, tensor parallel matrix multiplication may compute partial outputs that require all-reduce:



$$C = \sum_{p=0}^{P-1} A_p B_p.$$

Each device computes

$$C_p = A_p B_p,$$

then the result is

$$C = \text{allreduce}(C_0, C_1, \dots, C_{P-1}).$$

The search-based view is that communication placement and collective selection can also become part of the compiler search space.

## Common Misconceptions

### Misconception 1: A Small Primitive Set Means the System Cannot Express Real Models

A small primitive set does not mean the system is weak. Matrix multiplication, convolution, normalization, attention, and many other operations can be expressed as compositions of simpler tensor operations.

The cost is not expressiveness but performance. The naive primitive graph is slow until optimized.

### Misconception 2: Search-Based Compilation Is Just Ordinary Autotuning

Ordinary autotuning usually searches over a few parameters. Luminal searches over structural rewrites, algebraic transformations, loop transformations, and hardware intrinsic choices. This is a much larger and richer search space.

### Misconception 3: Mathematical Equivalence Guarantees Floating-Point Equivalence

Floating-point arithmetic is not exact real arithmetic. Rewrites such as reassociation may change numerical results. Correct search-based compilers must validate numerical tolerance or reason about error.

### **Misconception 4: Kernel Fusion Is Always Good**

Fusion reduces memory traffic, but it can increase register pressure, reduce occupancy, or make scheduling harder. In many elementwise cases fusion is beneficial, but in general it is a performance trade-off.

### **Misconception 5: The Compiler Should Search Everything**

Some transformations are deterministic and should not waste search budget. Buffer reuse and obvious launch queuing are examples where the compiler can apply known-good passes after search.

### **Misconception 6: Hardware Intrinsic Are Purely Backend Details**

Hardware intrinsics can change the algorithmic performance landscape. Whether to use tensor cores, asynchronous copies, or producer-consumer warp specialization may be a search decision, not just a final code generation detail.

### **Misconception 7: Dynamic Shapes Make Compilation Impossible**

Symbolic dimensions allow one kernel to support multiple shapes. The compiler can still profile representative shapes while keeping certain dimensions dynamic at runtime.

## **Practical Takeaways**

- A deep learning framework does not necessarily need thousands of primitive operations. Many high-level operations can be decomposed into a small tensor algebra basis.
- The naive primitive graph is not the execution target. It is the input to the compiler.
- Search can replace large amounts of handwritten heuristic compiler logic.
- E-graphs and equality saturation are useful for representing many equivalent programs compactly.
- Kernel fusion is best understood as moving intermediate values from global memory into registers or on-chip storage.
- Matrix multiplication is multiply plus reduce, but high performance requires tiling, reuse, and often tensor cores.
- Flash attention can be interpreted as a sequence of loop fusion, online softmax, tiling, and loop reordering transformations.

- Symbolic shapes are essential for practical serving workloads.
- Compile time is not always bad. If a model is deployed many times, expensive compilation can be worthwhile.
- As GPUs become more complex, search may become more valuable because fewer humans can reason manually about every hardware trade-off.

## Project or Experiment Ideas

### Experiment 1: Measure Elementwise Fusion

Implement two versions of

$$Y = \sin(\exp_2(X)).$$

Version one uses two separate CUDA or PyTorch operations. Version two uses a fused CUDA or Triton kernel. Measure runtime and estimate memory traffic.

Expected result:

$$\text{fused runtime} < \text{unfused runtime}$$

for large tensors because global memory traffic is reduced.

### Experiment 2: Naive Versus Tiled Matrix Multiplication

Implement the naive matrix multiplication kernel and the tiled shared memory kernel from these notes. Compare runtime for different  $M$ ,  $N$ , and  $K$ .

Measure:

- achieved GFLOP/s,
- global memory load efficiency,
- shared memory usage,
- occupancy,
- register count.

Use Nsight Compute to inspect whether the naive kernel is memory-bound and whether the tiled kernel improves reuse.

### Experiment 3: Symbolic Batch Size Benchmark

Write one kernel specialized for a fixed batch size and another kernel with runtime batch size. Compare performance across batch sizes such as

$$B \in \{1, 2, 4, 8, 16, 32, 64\}.$$

Study when shape specialization matters.

### Experiment 4: Mini Equality Saturation System

Build a small expression rewrite engine in Python for scalar algebra. Include rules such as:

$$x + 0 \rightarrow x,$$

$$x \cdot 1 \rightarrow x,$$

$$(x + y) + z \rightarrow x + (y + z),$$

$$x \cdot (y + z) \rightarrow xy + xz.$$

Then implement a simple cost model and extract the lowest-cost expression.

### Experiment 5: Online Softmax

Implement naive softmax and online softmax for a single row. Verify that both produce similar outputs within tolerance. Then extend it to blockwise attention.

Use the update equations:

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}}),$$

$$\ell_{\text{new}} = \ell_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}) + \ell_{\text{block}}.$$

### Experiment 6: Search Over Tile Sizes

Write a small autotuner for matrix multiplication that searches tile sizes:

$$T_M, T_N, T_K \in \{8, 16, 32, 64\}.$$

Measure runtime for each candidate and pick the fastest. This is a simplified version of the broader search philosophy.

## Final Mental Model

A conventional deep learning framework contains many operations, many kernels, many backends, and many special cases. Luminal’s search-based compiler approach asks a different question:

What if the framework core were tiny, neural networks were represented as primitive static tensor graphs, and performance came from searching the space of equivalent programs?

The primitive graph is intentionally simple and slow. It is not the final program. The compiler lowers it into an equality-saturation system, applies compact rewrite rules, builds a large e-graph of equivalent computations, extracts candidate kernels, validates numerical correctness, profiles them on real hardware, and keeps the fastest.

At the GPU level, most of the important wins come from controlling data movement:

less global memory traffic  $\Rightarrow$  higher arithmetic intensity  $\Rightarrow$  higher performance.

Fusion keeps intermediates in registers. Tiling reuses data in shared memory or registers. Tensor core rewrites map suitable multiply-reduce patterns onto specialized hardware. Online softmax enables flash-attention-style memory-efficient attention.

The deepest lesson is that search can absorb complexity. Instead of expecting human kernel authors or compiler engineers to manually predict every interaction among algebra, shapes, memory hierarchy, and hardware intrinsics, the compiler can generate many legal alternatives and let measurement decide.